



Tecnológico de Monterrey

Uso de framework o biblioteca de aprendizaje máquina para la implementación de una solución.

Algoritmo K-means

Michelle Rergis Novelo A01798576

9 de Septiembre del 2026

Módulo 2. Inteligencia artificial avanzada para la ciencia de datos.

Profesor: Jorge Adolfo Ramírez Uresti

1. Introducción y Objetivo

El aprendizaje no supervisado es una rama del aprendizaje máquina en la que los algoritmos exploran datos que no cuentan con una etiqueta o clase asignada de antemano. En lugar de aprender a predecir una salida conocida, el algoritmo busca patrones, similitudes o estructuras subyacentes en los datos. Dentro de esta rama, el clustering (agrupamiento) es la tarea de dividir un conjunto de instancias en grupos (clusters), de forma que los elementos dentro de un mismo grupo sean lo más parecidos posible entre sí, y lo más distintos posible de los elementos de otros grupos.

En este reporte se documenta la implementación de K-means utilizando la biblioteca scikit-learn de Python, su aplicación sobre el dataset Iris, y la evaluación de los resultados obtenidos mediante métricas de clustering (inertia, silhouette score, Adjusted Rand Index, Normalized Mutual Information) y, dado que el dataset utilizado cuenta con etiquetas reales, también mediante métricas de tipo clasificación (matriz de confusión, accuracy, precision, recall y F1-score), obtenidas tras mapear cada cluster a la clase real predominante en él.

1.1 ¿Qué es K-means?

K-means es un método de tipo particional y basado en centroides: cada cluster se representa mediante un punto central (centroide) y cada instancia del dataset se asigna al cluster cuyo centroide le resulte más cercano, usualmente utilizando la distancia euclidiana. El algoritmo itera entre la asignación de puntos a clusters y la actualización de los centroides hasta alcanzar convergencia.

1.2 Librerías utilizadas

La implementación se realizó en Python puro, ejecutable desde línea de comandos con el intérprete estándar (`python kmeans_es.py`). No se utilizó Jupyter Notebook ni ningún IDE específico; el único requisito es tener instaladas las librerías listadas a continuación.

Librería	Uso en el proyecto
scikit-learn	<code>make_blobs</code> (dataset), <code>train_test_split</code> , <code>StandardScaler</code> , <code>KMeans</code> , <code>PCA</code> , métricas (<code>silhouette</code> , <code>ARI</code> , <code>NMI</code> , matriz de confusión, <code>accuracy</code> , <code>F1</code>)
pandas	Manejo de datos en <code>DataFrame</code> / <code>Series</code> , nombres de variables y grupos
numpy	Operaciones numéricas y manejo de arreglos
matplotlib	Gráficas del codo, silueta y proyección PCA de los clusters
seaborn	Mapa de calor de la matriz de confusión
argparse	Parámetros de ejecución desde consola (<code>K</code> , tamaño de muestra, semilla, etc.)

Tabla 1. Librerías usadas.

El script acepta parámetros opcionales por consola (`--n-mue`, `--n-car`, `--n-grp`, `--k`, `--k-rango`, `--disp`, `--prueba`, `--semilla`, `--dir-sal`), lo que permite repetir el experimento con distintas configuraciones sin modificar el código.

2. Dataset: entrenamiento y prueba

2.1 Fuente y variables

Se utilizó un dataset sintético generado con la función `make_blobs` de `scikit-learn`, el cual crea grupos de datos (blobs) con distribución aproximadamente normal alrededor de centros aleatorios. Se eligió un dataset sintético porque permite conocer de antemano la etiqueta real de cada instancia (el grupo al que pertenece), lo cual hace posible evaluar objetivamente que tan bien K-means recupera esa estructura real, algo que no es posible con datos completamente no etiquetados del mundo real.

Configuración del dataset generado:

- 1500 muestras (instancias) en total.
- 6 variables numéricas (Var_1 a Var_6).
- 4 grupos reales (Grupo_0 a Grupo_3), usados únicamente como referencia para evaluar, no durante el entrenamiento de K-means.
- Dispersión (cluster_std) de 2.5, para generar cierto traslape entre grupos y que el problema no sea trivial.
- Semilla aleatoria fija (random_state=42) para asegurar resultados reproducibles.

2.2 Partición entrenamiento / prueba

El dataset se dividió en un conjunto de entrenamiento y uno de prueba de forma estratificada (misma proporción de grupos en ambos subconjuntos):

Conjunto	Muestras	Proporción	Uso
Entrenamiento	1200	80%	Ajustar centroides con <code>KMeans.fit()</code>
Prueba	300	20%	Evaluar con <code>KMeans.predict()</code> y comparar contra el grupo real

Tabla 2. División del dataset.

Antes de entrenar el modelo, las 6 variables se estandarizaron con `StandardScaler` (media 0, desviación estándar 1), ajustado únicamente con el conjunto de entrenamiento y aplicado después al conjunto de prueba. Esto es importante porque K-means se basa en distancias, y variables con escalas muy distintas dominarían el cálculo de distancia si no se normalizan.

3. Flujo general del algoritmo

El siguiente diagrama resume el flujo completo implementado en el script `kmeans_es.py`, desde la generación de los datos hasta la predicción de nuevos puntos:

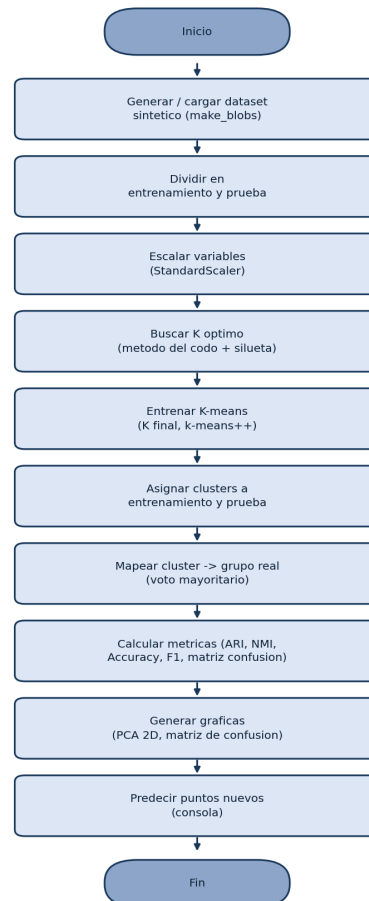


Figura 1. Diagrama de flujo de la implementación de K-means.

3.1 Etapas

- **Generar / cargar dataset:** se crean 1500 muestras sintéticas con 4 grupos reales mediante `make_blobs`.
- **Dividir en entrenamiento y prueba:** 80% / 20% de forma estratificada.
- **Escalar variables:** `StandardScaler` ajustado en entrenamiento y aplicado igual en prueba.
- **Buscar K óptimo:** se entrena un modelo K-means para cada K entre 2 y 8, registrando la inercia (WCSS) y el coeficiente de silueta de cada uno; con ello se generan las gráficas del método del codo y de silueta.

- **Entrenar K-means:** se ajusta el modelo final con el K elegido, inicialización k-means++, 10 inicializaciones distintas (n_init=10) y máximo 300 iteraciones.
- **Asignar clusters:** se obtienen las etiquetas de cluster para entrenamiento (labels_) y se predicen para prueba (predict()).
- **Mapear cluster — grupo real:** como K-means no conoce el nombre de los grupos reales, a cada cluster se le asigna el grupo real más frecuente entre sus miembros de entrenamiento (voto mayoritario), únicamente con fines de evaluación.
- **Calcular métricas:** silhouette, Adjusted Rand Index (ARI), Normalized Mutual Information (NMI), matriz de confusión, accuracy y F1-score.
- **Generar gráficas:** proyección PCA a 2 dimensiones de los clusters (entrenamiento y prueba) y mapa de calor de la matriz de confusión.
- **Predecir puntos nuevos:** se muestra en consola, a manera de demostración, la asignación de cluster/grupo para instancias tomadas del conjunto de prueba.

4. Conceptos y justificación del algoritmo

4.1 Distancia Euclidiana

K-means asigna cada punto al centroide más cercano usando la distancia euclidiana, que corresponde a la distancia en línea recta entre dos puntos en un espacio de N dimensiones. Es la métrica estándar del algoritmo porque es diferenciable y se relaciona directamente con la minimización de la varianza dentro de cada cluster.

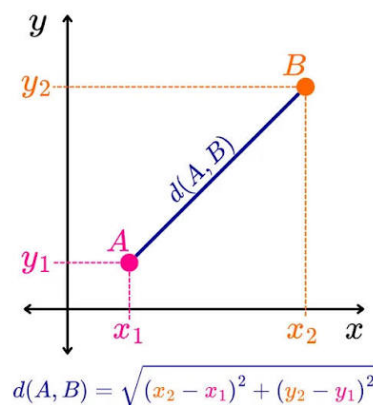


Figura 2. Distancia Euclidiana entre dos puntos.

4.2 Inercia (WCSS)

La inercia, también llamada WCSS (Within-Cluster Sum of Squares), es la suma de las distancias al cuadrado de cada punto a su centroide. K-means busca minimizar esta cantidad: entre menor sea la inercia, más compactos son los clusters. Sin embargo, la inercia siempre disminuye al aumentar K, por lo que no puede usarse sola para elegir el número de clusters.

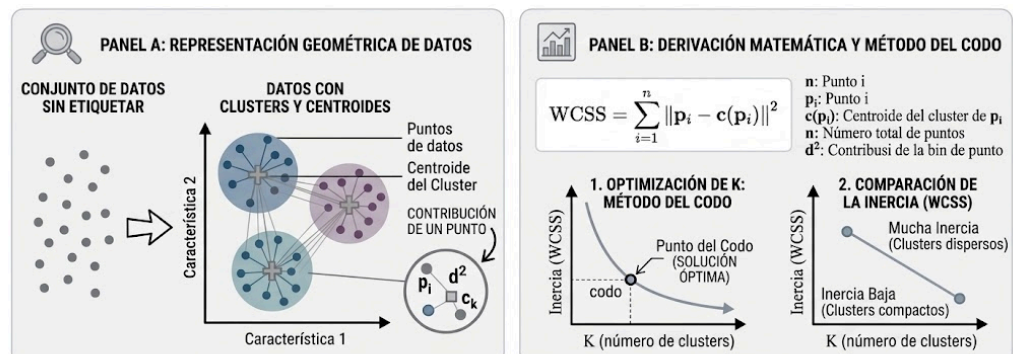


Figura 3. Diagrama general de cómo funciona Inercia WCSS (imagen generada con Gemini).

4.3 Método del codo

Consiste en graficar la inercia contra distintos valores de K y buscar el punto donde la reducción de la inercia se vuelve marginal (el 'codo' de la curva). En este proyecto, la gráfica del codo muestra una caída pronunciada hasta K=4, y a partir de ahí la mejora es mucho menor, lo cual coincide con el número real de grupos generados.

4.4 Coeficiente de silueta

Mide, para cada punto, que tan similar es a su propio cluster comparado con el cluster más cercano distinto al suyo. Su valor va de -1 a 1: valores altos indican clusters bien separados y compactos. En este proyecto, la silueta señala un máximo en K=3, muy cercano al valor obtenido en K=4, lo que evidencia que ambos métodos deben interpretarse en conjunto y no de forma aislada.

Para cada punto i:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \quad \text{si } |Ci| > 1, \quad s(i) = 0 \quad \text{si } |Ci| = 1$$

Donde:

a(i) = distancia promedio de **i** a todos los demás puntos de **su propio cluster** (cohesión intra-cluster):

$$a(i) = \frac{1}{|C_i|-1} \sum_{j \in C_i, j \neq i} d(i, j)$$

b(i) = distancia promedio mínima de **i** a los puntos del **cluster vecino más cercano** (separación inter-cluster):

$$b(i) = \min_{k \neq i} \frac{1}{|C_k|} \sum_{j \in C_k} d(i, j)$$

4.5 Adjusted Rand Index (ARI) y Normalized Mutual Information (NMI)

Ambas métricas comparan la partición generada por K-means contra la partición real (los grupos verdaderos del dataset sintético), penalizando coincidencias por azar. Un valor de 1 indica coincidencia perfecta entre clusters y grupos reales; un valor cercano a 0 indica una partición prácticamente aleatoria.

4.6 Matriz de confusión, accuracy y F1-score

Dado que en este experimento si se conoce el grupo real de cada instancia, es posible mapear cada cluster a su grupo más frecuente y tratar el problema como si fuera de clasificación, calculando una matriz de confusión, accuracy y F1-score macro. Esto no es un procedimiento estándar de un problema de clustering en producción (donde no se conocen las etiquetas reales), pero es una práctica válida y común para validar y demostrar el desempeño del algoritmo cuando se dispone de una verdad base (dataset sintético), como es el caso de este proyecto.

6.7 Justificación de la configuración elegida

- **init='k-means++'**: mejora la inicialización de los centroides respecto a una selección aleatoria simple, reduciendo el riesgo de converger a un mínimo local pobre.
- **n_init=10**: el algoritmo se ejecuta 10 veces con distintas inicializaciones y se conserva la de menor inercia, haciendo el resultado más estable.

- ***max_iter=300***: límite de iteraciones de asignación/actualización de centroides, suficiente para asegurar convergencia en este problema.
- ***random_state=42***: semilla fija para reproducibilidad de resultados.
- ***Escalado previo con StandardScaler***: evita que variables con mayor magnitud dominen el cálculo de distancia euclidiana.

4. Resultados obtenidos

Se entrenó el modelo con el conjunto de entrenamiento (1200 muestras) y se evaluó sobre el conjunto de prueba (300 muestras), el cual el modelo nunca vio durante el ajuste de los centroides. A continuación se presentan los resultados para dos configuraciones de K: el valor sugerido automáticamente por el coeficiente de silueta ($K=3$) y el número real de grupos del dataset ($K=4$), con el fin de mostrar el impacto de la selección de K en el desempeño final.

4.1 Búsqueda de K (método del codo y silueta)

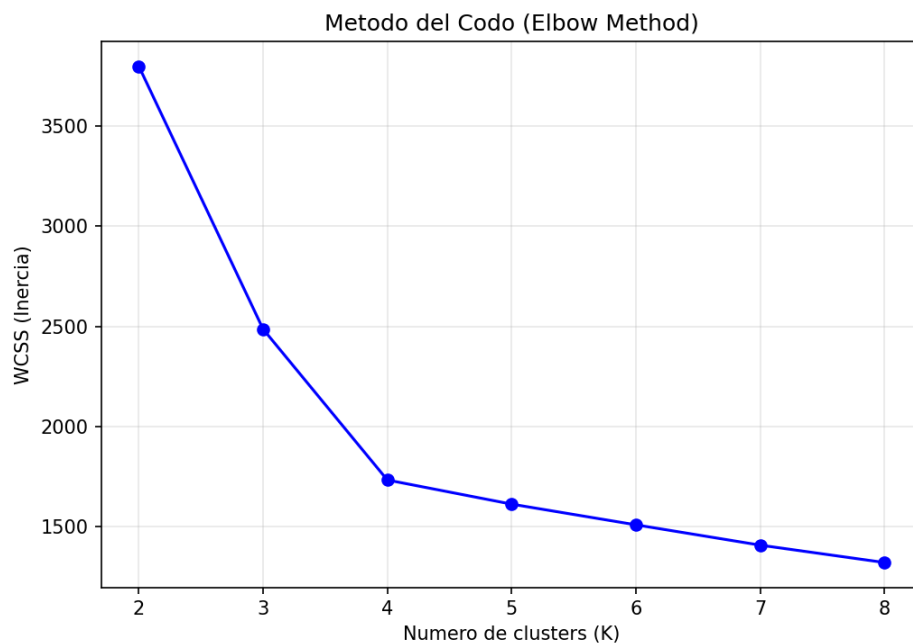


Figura 4. Método del codo: la inercia (WCSS) cae fuertemente hasta $K=4$

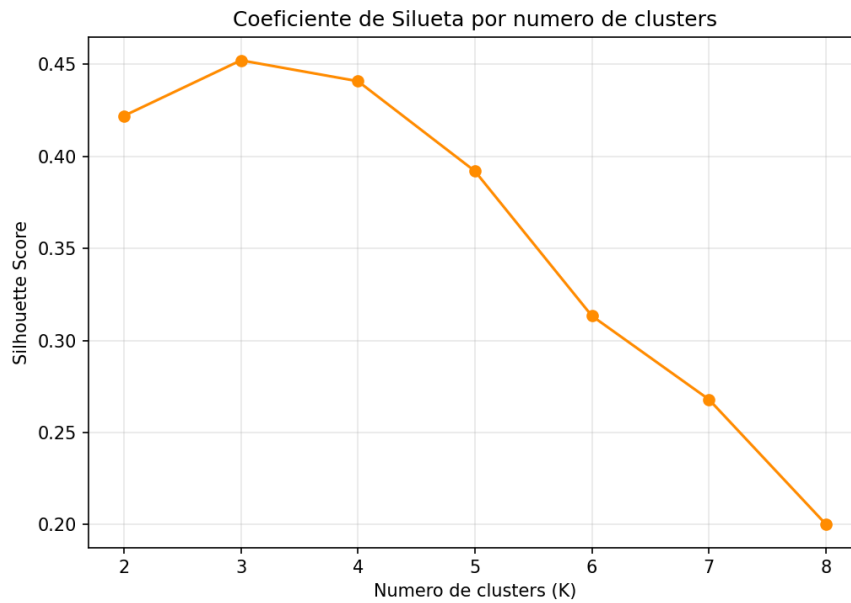


Figura 5. Coeficiente de silueta: el máximo se alcanza en $K=3$, muy cerca de $K=4$.

4.2 Comparación de métricas según K

Métrica	K = 3 (sugerido por silueta)	K = 4 (número real de grupos)
Inercia (WCSS)	2486.275	1732.497
Silhouette (entrenamiento)	0.4522	0.4409
Silhouette (prueba)	0.4486	0.4458
Adjusted Rand Index (ARI)	0.7003	0.9911
Normalized Mutual Info (NMI)	0.8291	0.9872
Accuracy (grupo mapeado)	0.7467	0.9967
F1-Score macro	0.6624	0.9967

Tabla 3. Comparación de métricas entre $K=3$ y $K=4$.

4.3 Visualización de clusters (proyección PCA)

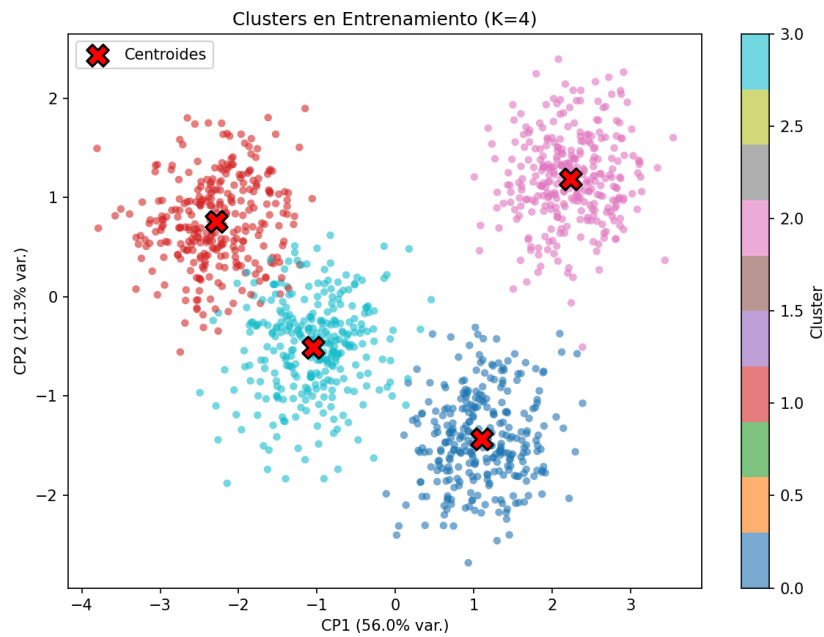


Figura 6. Clusters obtenidos en entrenamiento con $K=4$, proyectados a 2D con PCA (77.3% de varianza explicada entre ambos componentes).

4.4 Matrices de confusión

Con $K=3$, dos de los cuatro grupos reales (Grupo_2 y Grupo_3) quedan mezclados dentro de un mismo cluster, ya que no hay suficientes clusters para separarlos; por ello la columna de Grupo_3 aparece vacía en la matriz de confusión.

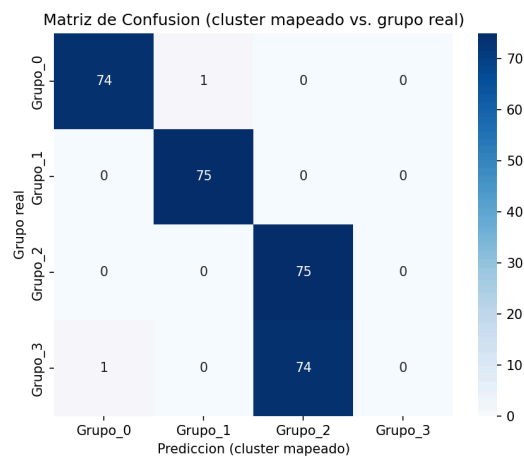


Figura 7. Matriz de confusión con $K=3$ (cluster mapeado vs. grupo real, conjunto de prueba).

Con $K=4$ (número real de grupos), la matriz de confusión es prácticamente diagonal: cada grupo real queda correctamente representado por un cluster distinto, con un único error de clasificación en 300 muestras de prueba.

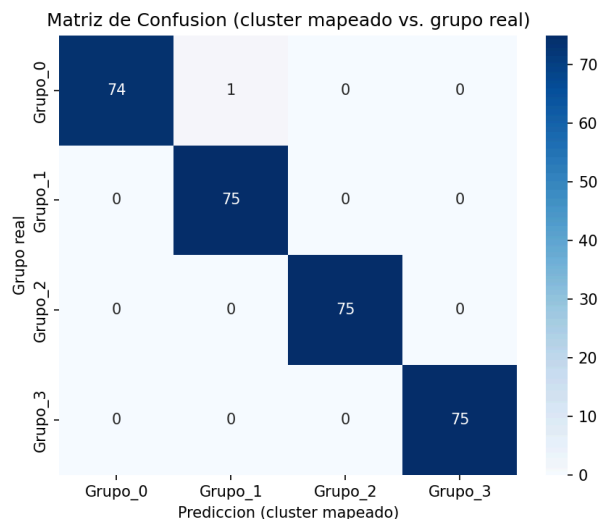


Figura 8. Matriz de confusión con $K=4$ (cluster mapeado vs. grupo real, conjunto de prueba).

4.5 Ejemplo de predicciones en consola

El script también ejemplifica el uso del modelo ya entrenado para predecir el cluster de nuevas observaciones (tomadas aquí del conjunto de prueba), imprimiendo en consola el cluster asignado y el grupo real correspondiente según el mapeo por voto mayoritario. Un extracto de esta salida (ejecución con $K=4$) es:

[PREDICCION] Punto [-4.65, -0.90, 1.54, -8.89, -7.29, -4.06] -> Cluster 3 -> Grupo_3

5. Análisis y conclusión del desempeño

El método del codo y el coeficiente de silueta no señalaron de forma unánime el número real de grupos: el codo sugiere una caída clara hasta $K=4$, mientras que la silueta obtiene su valor máximo en $K=3$ (0.4522), muy cercano al de $K=4$ (0.4409). Esto ilustra una limitación práctica conocida de K-means: cuando existe traslape entre clusters (como se generó deliberadamente con `cluster_std=2.5`), las heurísticas para elegir K pueden sugerir un valor distinto al número real de grupos subyacentes.

Al forzar $K=4$ (el número real de grupos del dataset), el desempeño mejora de forma sustancial en todas las métricas que comparan contra la verdad base: el ARI sube de 0.7003 a 0.9911, el NMI de 0.8291 a 0.9872, y la accuracy de 0.7467 a 0.9967. La matriz de confusión con $K=4$ es casi perfectamente diagonal (299 de 300 muestras de prueba correctamente asignadas a su grupo), mientras que con $K=3$ dos grupos reales terminan fusionados en un mismo cluster.

Además, la inercia y el silhouette score de entrenamiento y prueba resultaron muy similares entre sí en ambas configuraciones (por ejemplo, silhouette de 0.4522 en entrenamiento vs. 0.4486 en prueba con $K=3$), lo que indica que el modelo generaliza bien: los centroides aprendidos con el 80% de los datos describen adecuadamente al 20% restante que nunca fue usado para ajustarlos.

Como conclusión, K-means demostró ser un algoritmo simple, rápido y efectivo para agrupar datos con forma esférica y tamaños de cluster similares, como los del dataset sintético utilizado. Su principal debilidad práctica radica en la necesidad de definir K de antemano: las técnicas del codo y de silueta son útiles como guía, pero no siempre son concluyentes cuando los grupos no están perfectamente separados, por lo que en un problema real conviene combinarlas con conocimiento del dominio antes de fijar el valor final de K .

6. Referencias

Russell, S. & Norvig, P. (2010). Artificial Intelligence: A Modern Approach. Third Edition. Prentice Hall.

Mitchell, T. (1997). Machine Learning. McGraw-Hill.

Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830. Documentación oficial de scikit-learn: <https://scikit-learn.org/stable/modules/clustering.html#k-means>

Armijo, P. (2026). Aprendizaje No Supervisado. Tema 8, TC2034 - Modelación del Aprendizaje con Inteligencia Artificial. Tecnológico de Monterrey.